WEEK- 4

1.PROBLEM STATEMENT:

You are given a string s of even length. Split this string into two halves of equal lengths, and let a be the first half and b be the second half.

Two strings are **alike** if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'). Notice that s contains uppercase and lowercase letters.

Return true *if* a *and* b *are* ***alike***. Otherwise, return false.

Example 1:

Input: s = "book"

Output: true

Constraints:

- 2 <= s.length <= 1000
- s.length is even.
- s consists of uppercase and lowercase letters.

SOURCE CODE:

```
public class Solution {
    public boolean halvesAreAlike(String s) {
        Set<Character> vowels = new HashSet<>();
        vowels.add('a'); vowels.add('e'); vowels.add('i'); vowels.add('o'); vowels.add('u');
        vowels.add('A'); vowels.add('E'); vowels.add('I'); vowels.add('O'); vowels.add('U');

        int vowelsCount = 0;
        int midIndex = s.length() / 2;
```

```
        for (int i = 0; i < midIndex; i++) {
            char charA = s.charAt(i);
            char charB = s.charAt(midIndex + i);
            if (vowels.contains(charA)) vowelsCount++;
            if (vowels.contains(charB)) vowelsCount--;
        }

        return vowelsCount == 0;
    }
}
```

OUTPUT:



2.PROBLEM STATEMENT:

Lapindrome is defined as a string which when split in the middle, gives two halves having the same characters and same frequency of each character. If there are odd number of characters in the string, we ignore the middle character and check for lapindrome. For example gaga is a lapindrome, since the two halves ga and ga have the same characters with same frequency.
Also, abccab, rotor and xyzxy are a few examples of lapindromes. Note that abbaab is NOT a lapindrome. The two halves contain the same characters but their frequencies do not match.
Your task is simple. Given a string, you need to tell if it is a lapindrome.

Input:

First line of input contains a single integer T, the number of test cases.
Each test is a single line containing a string S composed of only lowercase English alphabet.
Output:

For each test case, output on a separate line: "YES" if the string is a lapindrome and "NO" if it is not.

Constraints:

- $1 \leq T \leq 100$
- $2 \leq |S| \leq 1000$, where $|S|$ denotes the length of S

SOURCE CODE:

```
import java.util.Scanner;

import java.util.Arrays;


public class Main {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      if (sc.hasNextInt()) {
```

```
        int t = sc.nextInt();
        while (t-- > 0) {
            String s = sc.next();
            int n = s.length();
            int[] leftFreq = new int[26];
            int[] rightFreq = new int[26];
            for (int i = 0; i < n / 2; i++) {
                leftFreq[s.charAt(i) - 'a']++;
            }
            int startRight = (n % 2 == 0) ? n / 2 : (n / 2) + 1;
            for (int i = startRight; i < n; i++) {
                rightFreq[s.charAt(i) - 'a']++;
            }
            if (Arrays.equals(leftFreq, rightFreq)) {
                System.out.println("YES");
            } else {
                System.out.println("NO");
            }
        }
        sc.close();
    }
}
```

OUTPUT:

Sample Input

```
6
gaga
abcde
rotor
xyzxy
abbaab
ababc
```

Your Output

```
YES
NO
YES
YES
NO
NO
```

3.PROBLEM STATEMENT:

Alice and Bob each created a reviewer rates the two challenges, awarding points on a scale from 1 to 100 for three categories: problem clarity, originality, and difficulty.

The rating for Alice's challenge is the triplet $a = (a[0], a[1], a[2])$, and the rating for Bob's challenge is the triplet $b = (b[0], b[1], b[2])$.

Your task is to calculate their comparison points by comparing each category:

- If $a[i] > b[i]$, then Alice is awarded 1 point.
- If $a[i] < b[i]$, then Bob is awarded 1 point.
- If $a[i] = b[i]$, then neither person receives a point.

Function Description

Complete the compareTriplets function.

Parameters:

- int a[3]: Alice's challenge rating
- int b[3]: Bob's challenge rating

Returns:

- int[2]: The first element is Alice's score and the second is Bob's score.

Input Format

- The first line contains 3 space-separated integers, $a[0]$, $a[1]$, and $a[2]$, the respective values in triplet a.
- The second line contains 3 space-separated integers, $b[0]$, $b[1]$, and $b[2]$, the respective values in triplet b.

Constraints

- $1 \leq a[i] \leq 100$
- $1 \leq b[i] \leq 100$

SOURCE CODE:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
class Result {
   public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b)
{
       int alicePoints = 0;
       int bobPoints = 0;
       for (int i = 0; i < 3; i++) {
          if (a.get(i) > b.get(i)) {
             alicePoints++;
          } else if (a.get(i) < b.get(i)) {
             bobPoints++;
          }
       }
```

```
        return Arrays.asList(alicePoints, bobPoints);
    }


}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
        List<Integer> a = Stream.of(bufferedReader.readLine().replaceAll("\\s+$",
"").split(" "))
            .map(Integer::parseInt)
            .collect(toList());
        List<Integer> b = Stream.of(bufferedReader.readLine().replaceAll("\\s+$",
"").split(" "))
            .map(Integer::parseInt)
            .collect(toList());
        List<Integer> result = Result.compareTriplets(a, b);
        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining(" "))
            + "\n");
        bufferedReader.close();
        bufferedWriter.close();
    }
```

}

OUTPUT:

Input (stdin)

```
5 6 7
3 6 10
```

Your Output (stdout)

```
1 1
```

Expected Output

```
1 1
```

4.PROBLEM STATEMENT:

Given an integer array nums, return true if any value appears **at least twice** in the array, and return false if every element is distinct.

Example 1:

Input: nums = [1,2,3,1]

Output: true

Explanation:

The element 1 occurs at the indices 0 and 3.

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$

SOURCE CODE:

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                if (nums[i] == nums[j])
                    return true;
            }
        }
        return false;
    }
```

}

OUTPUT:

Input

nums =

[1,2,3,1]

Output

true

Expected

true

5.PROBLEM STATEMENT:

Time Conversion:

Given a time in 12-hour AM/PM format, convert it to military (24-hour) time.

Note:

- 12:00:00AM on a 12-hour clock is 00:00:00 on a 24-hour clock.
- 12:00:00PM on a 12-hour clock is 12:00:00 on a 24-hour clock.

  Example :

  - Input: $s =$ '12:01:00PM'
  - Return: '12:01:00'

Function Description

Complete the timeConversion function in the editor.

- Parameters:
  - string s: a time in 12-hour clock format.
- Returns:
  - string: the time in 24-hour clock format.

Input & Constraints

- Input Format: A single string s representing a time in 12-hour clock format (e.g., hh:mm:ssAM or hh:mm:ssPM).
- Constraints: All input times are valid.

SOURCE CODE:

```
import java.io.*;

import java.math.*;
```

```
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
class Result {
    public static String timeConversion(String s) {
        String period = s.substring(8);
        int hour = Integer.parseInt(s.substring(0, 2));
        String restOfTime = s.substring(2, 8);
        if (period.equals("AM")) {
            if (hour == 12) {
                hour = 0;
            }
        } else {
            if (hour != 12) {
                hour += 12;
            }
        }
        return String.format("%02d%s", hour, restOfTime);
    }
```

```
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

        String result = Result.timeConversion(s);

        bufferedWriter.write(result);
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:

Input (stdin)

```
07:05:45PM
```

Your Output (stdout)

```
19:05:45
```

Expected Output

```
19:05:45
```

6.PROBLEM STATEMENT:

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: nums = [0,1,0,3,12]

Output: [1,3,12,0,0]

Constraints:

- $1 <= nums.length <= 10^4$
- $-2^{31} <= nums[i] <= 2^{31} - 1$

SOURCE CODE:

```
class Solution {
    public void moveZeroes(int[] nums) {
        int left = 0;
        for (int right = 0; right < nums.length; right++) {
            if (nums[right] != 0) {
                int temp = nums[right];
                nums[right] = nums[left];
                nums[left] = temp;
                left++;
            }
        }
    }
```

}

OUTPUT:

Input

nums =

```
[0,1,0,3,12]
```

Output

```
[1,3,12,0,0]
```

Expected

```
[1,3,12,0,0]
```

7.PROBLEM STATEMENT:

To solve the Diagonal Difference challenge on HackerRank, you need to iterate through the rows of a square matrix ($n \times n$), sum the elements along both the primary and secondary diagonals, and return the absolute difference between the two sums.

Algorithm Breakdown

- Primary Diagonal: Elements where the row index equals the column index (arr[i][i]).
- Secondary Diagonal: Elements where the column index is $n - 1 - i$ (arr[i][n - 1 - i]).
- Absolute Difference: |primary_sum − secondary_sum| using standard mathematical utilities

SOURCE CODE:

```
import java.io.*;

import java.math.*;

import java.security.*;

import java.text.*;

import java.util.*;

import java.util.concurrent.*;

import java.util.regex.*;

class Result {

    public static int diagonalDifference(List<List<Integer>> arr) {

        int primarySum = 0;

        int secondarySum = 0;

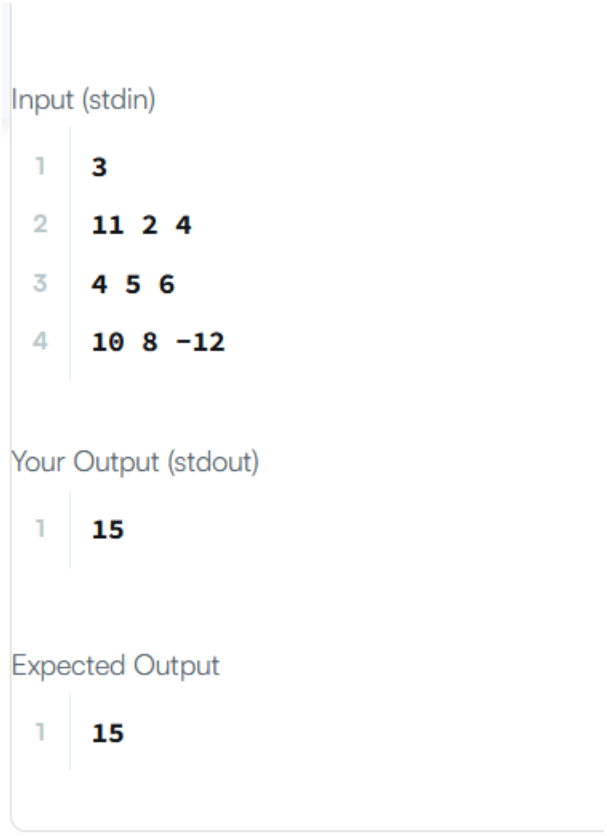
        int n = arr.size();

        for (int i = 0; i < n; i++) {
```

```
            primarySum += arr.get(i).get(i);
            secondarySum += arr.get(i).get(n - 1 - i);
        }
        return Math.abs(primarySum - secondarySum);
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
        int n = Integer.parseInt(bufferedReader.readLine().trim());
        List<List<Integer>> arr = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            String[] arrRowTempItems =
bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");
            List<Integer> arrRowItems = new ArrayList<>()
            for (int j = 0; j < n; j++) {
                int arrItem = Integer.parseInt(arrRowTempItems[j]);
                arrRowItems.add(arrItem);
            }
            arr.add(arrRowItems);
        }
        int result = Result.diagonalDifference(arr);
        bufferedWriter.write(String.valueOf(result));
```

```
        bufferedWriter.newLine();
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

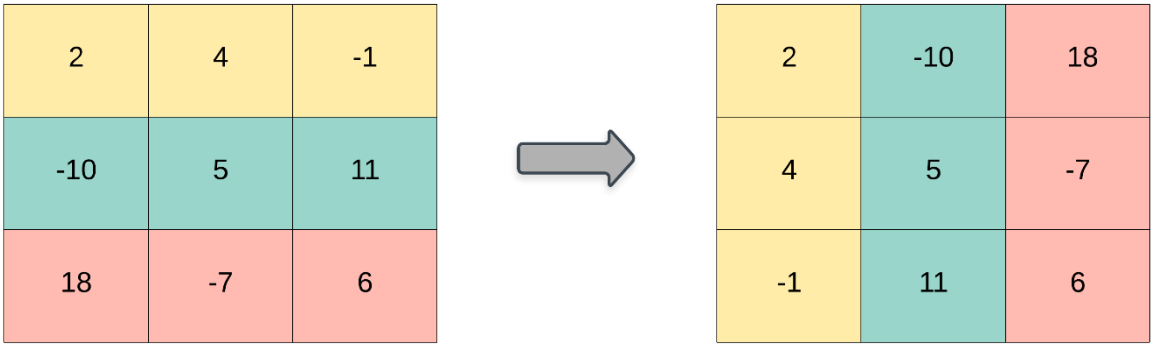
OUTPUT:



8.PROBLEM STATEMENT:

Given a 2D integer array matrix, return *the transpose of* matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.



Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [[1,4,7],[2,5,8],[3,6,9]]

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 1000
- 1 <= m * n <= 10^5
- -10^9 <= matrix[i][j] <= 10^9

SOURCE CODE:

```
class Solution {
    public int[][] transpose(int[][] matrix) {
```

```
        int rows = matrix.length;
        int cols = matrix[0].length;
        int[][] result = new int[cols][rows];
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                result[j][i] = matrix[i][j];
            }
        }
        return result;
    }
}
```

OUTPUT:

Input

```
matrix =
[[1,2,3],[4,5,6],[7,8,9]]
```

Output

```
[[1,4,7],[2,5,8],[3,6,9]]
```

Expected

```
[[1,4,7],[2,5,8],[3,6,9]]
```

9.PROBLEM STATEMENT:

Given a m x n matrix mat and an integer k, return *a matrix* answer *where each* answer[i][j] *is the sum of all elements* mat[r][c] *for*:

- i - k <= r <= i + k,
- j - k <= c <= j + k, and
- (r, c) is a valid position in the matrix.

Example 1:

Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 1

Output: [[12,21,16],[27,45,33],[24,39,28]]

Constraints:

- m == mat.length
- n == mat[i].length
- 1 <= m, n, k <= 100
- 1 <= mat[i][j] <= 100

SOURCE CODE:

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {
        int m = mat.length;
        int n = mat[0].length;
        int[][] prefix = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
```

```
            prefix[i + 1][j + 1] =
                    mat[i][j]
                    + prefix[i][j + 1]
                    + prefix[i + 1][j]
                    - prefix[i][j];
            }
        }
        int[][] result = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++)
                int r1 = Math.max(0, i - k);
                int r2 = Math.min(m - 1, i + k);
                int c1 = Math.max(0, j - k);
                int c2 = Math.min(n - 1, j + k);
                result[i][j] =
                        prefix[r2 + 1][c2 + 1]
                        - prefix[r1][c2 + 1]
                        - prefix[r2 + 1][c1]
                        + prefix[r1][c1];
            }
        }
        return result;
    }
}
```

OUTPUT:

Input

mat =

```
[[1,2,3],[4,5,6],[7,8,9]]
```

k =

```
1
```

Output

```
[[12,21,16],[27,45,33],[24,39,28]]
```

Expected

```
[[12,21,16],[27,45,33],[24,39,28]]
```

10.PROBLEM STATEMENT:

The Matrix Layer Rotation problem asks you to rotate a 2D matrix of size $m \times n$ in an anti-clockwise direction r times and print the resulting matrix.

Key Rules & Requirements

- Layer-by-Layer Rotation: The matrix is rotated in concentric layers (rings). Each element shifts anti-clockwise along its specific ring by r positions.
- Dimensions: You are given m rows and n columns. The minimum of m and n is guaranteed to be an even number.
- Output Format: Print the transformed 2D matrix row-by-row, with elements in each row separated by spaces. Return nothing.

Constraints & Notes

- r can be large, so rotating layer elements one step at a time will be too slow. You should calculate effective rotations using modulo arithmetic based on each layer's perimeter length: $r_{\text{eff}} = r \pmod{\text{layer_length}}$.

SOURCE CODE:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void matrixRotation(List<List<Integer>> matrix, int r) {
        int m = matrix.size();
        int n = matrix.get(0).size();
        int numLayers = Math.min(m, n) / 2;
        for (int layer = 0; layer < numLayers; layer++) {
```

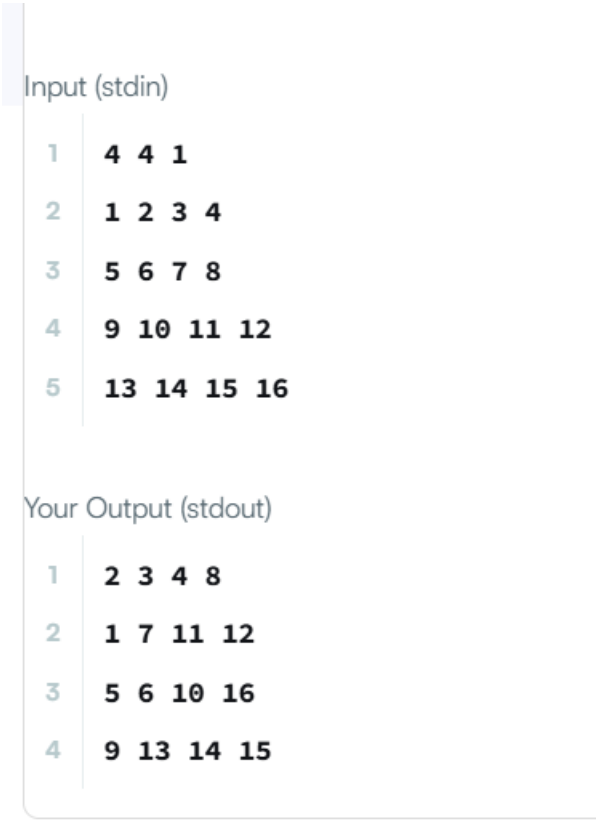
```
        List<Integer> elements = new ArrayList<>();

        for (int j = layer; j < n - layer; j++) {
            elements.add(matrix.get(layer).get(j));
        }
        for (int i = layer + 1; i < m - layer; i++) {
            elements.add(matrix.get(i).get(n - 1 - layer));
        }
        for (int j = n - 2 - layer; j >= layer; j--) {
            elements.add(matrix.get(m - 1 - layer).get(j));
        }
        for (int i = m - 2 - layer; i > layer; i--) {
            elements.add(matrix.get(i).get(layer));
        }
        int len = elements.size();
        int shift = r % len;
        int index = shift;
        for (int j = layer; j < n - layer; j++) {
            matrix.get(layer).set(j, elements.get(index % len));index++;
        }
        for (int i = layer + 1; i < m - layer; i++) {
      matrix.get(i).set(n - 1 - layer, elements.get(index % len));index++;
        }
        for (int j = n - 2 - layer; j >= layer; j--) {
matrix.get(m - 1 - layer).set(j, elements.get(index % len));   index++;
```

```
            }
            for (int i = m - 2 - layer; i > layer; i--) {
                matrix.get(i).set(layer, elements.get(index % len));
                index++;
            }
        }
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                sb.append(matrix.get(i).get(j));
                if (j < n - 1) sb.append(" ");
            }           sb.append("\n");
        }
        System.out.print(sb.toString());
    }
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
    String[] firstMultipleInput = bufferedReader.readLine().trim().split(" ");
        int m = Integer.parseInt(firstMultipleInput[0]);
        int n = Integer.parseInt(firstMultipleInput[1]);
        int r = Integer.parseInt(firstMultipleInput[2]);
        List<List<Integer>> matrix = new ArrayList<>();
        for (int i = 0; i < m; i++) {
       String[] matrixRowItems = bufferedReader.readLine().trim().split(" ");
```

```
            List<Integer> matrixRow = new ArrayList<>();
            for (int j = 0; j < n; j++) {
                matrixRow.add(Integer.parseInt(matrixRowItems[j]));
            }
            matrix.add(matrixRow);
        }
        matrixRotation(matrix, r);
        bufferedReader.close();
    }
}
```

OUTPUT:



WEEK -5

1.PROBLEM STATEMENT:

For two strings A and B, we define the similarity of the strings to be the length of the longest prefix common to both strings. For example, the similarity of strings "abc" and "abd" is 2, while the similarity of strings "aaa" and "aaab" is 3.

Given a string S, calculate the sum of similarities of S with each of its suffixes.

Input Format

- The first line contains the number of test cases, t.
- Each of the next t lines contains a string to process, s.

Constraints

- $1 \leq t \leq 10$
- $1 \leq |s| \leq 100{,}000$
- s is composed of lowercase ASCII characters (a-z).

SOURCE CODE:

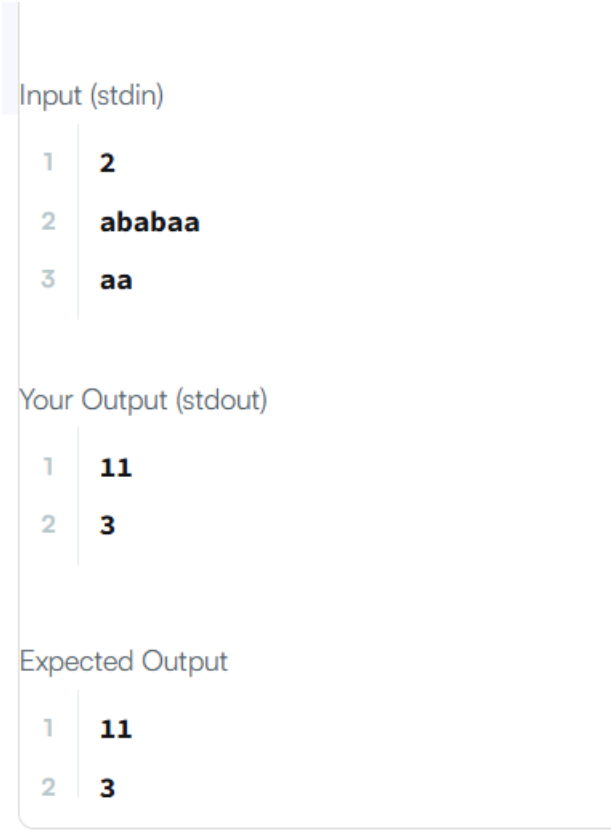
```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
public class Solution {
    public static long calculateStringSimilarity(String s) {
        int n = s.length();
        int[] z = new int[n];
```

```
        z[0] = n;
        long totalSimilarity = n;

        int l = 0, r = 0;
        for (int i = 1; i < n; i++) {
            if (i <= r) {
                z[i] = Math.min(r - i + 1, z[i - l]);
            }
            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
                z[i]++;
            }
            if (i + z[i] - 1 > r) {
                l = i;
                r = i + z[i] - 1;
            }
            totalSimilarity += z[i];
        }
        return totalSimilarity;
    }
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new
InputStreamReader(System.in));
        String tStr = reader.readLine();
        if (tStr == null || tStr.trim().isEmpty()) return;
        int t = Integer.parseInt(tStr.trim());
```

```
        StringBuilder result = new StringBuilder();
        while (t-- > 0) {
            String s = reader.readLine().trim();
            result.append(calculateStringSimilarity(s)).append("\n");
        }
        System.out.print(result);
    }
}
```

OUTPUT:



2.PROBLEM STATEMENT:

Given a string s, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

Input: s = "abab"

Output: true

Explanation: It is the substring "ab" twice.

Constraints:

- 1 <= s.length <= 10^4
- s consists of lowercase English letters.

SOURCE CODE:

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {
        int n = s.length();
        int[] lps = new int[n];
        for (int i = 1, j = 0; i < n; i++) {
            while (j > 0 && s.charAt(i) != s.charAt(j)) {
                j = lps[j - 1];
            }
            if (s.charAt(i) == s.charAt(j)) {
                j++;
```

```
            }
            lps[i] = j;
        }
        int len = lps[n - 1];
        return len > 0 && n % (n - len) == 0;
    }
}
```

OUTPUT:



3.PROBLEM STATEMENT:

Given an array of integers, find the maximum possible sum among:

1. All non-empty subarrays (a contiguous subsequence in the array).
2. All non-empty subsequences (any subset of elements keeping order, not necessarily contiguous).

Function Description

Complete the maxSubarray function.

- Input Parameter: int arr[n] — an array of integers.
- Returns: int[2] — an array containing [max_subarray_sum, max_subsequence_sum].

Constraints & Rules

- $1 \leq t \leq 10$ (number of test cases)
- $1 \leq n \leq 10^5$
- $-10^4 \leq arr[i] \leq 10^4$
- Both the subarray and subsequence must contain **at least one element** (empty sets are not allowed).
- If all elements in the array are negative, both values will equal the single largest (least negative) element in the array.

SOURCE CODE:

```
import java.io.*;

import java.util.*;
```

```
import java.util.stream.*;
import static java.util.stream.Collectors.toList;


class Result {
    public static List<Integer> maxSubarray(List<Integer> arr) {
        int maxSubarraySum = arr.get(0);
        int currentSubarraySum = arr.get(0);
        int maxSubsequenceSum = 0;
        int maxElement = arr.get(0);
        boolean hasPositive = false;
        for (int num : arr) {
            if (num > maxElement) {
                maxElement = num;
            }
            if (num > 0) {
                maxSubsequenceSum += num;
                hasPositive = true;
            }
        }
        for (int i = 1; i < arr.size(); i++) {
            currentSubarraySum = Math.max(arr.get(i), currentSubarraySum +
arr.get(i));
            maxSubarraySum = Math.max(maxSubarraySum,
currentSubarraySum);
        }
```

```
        if (!hasPositive) {
            maxSubsequenceSum = maxElement;
        }

        return Arrays.asList(maxSubarraySum, maxSubsequenceSum);
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
        int t = Integer.parseInt(bufferedReader.readLine().trim());
        IntStream.range(0, t).forEach(tItr -> {
            try {
                int n = Integer.parseInt(bufferedReader.readLine().trim());
                List<Integer> arr =
Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
                    .map(Integer::parseInt)
                    .collect(toList());
                List<Integer> result = Result.maxSubarray(arr);
                bufferedWriter.write(
                    result.stream()
                        .map(Object::toString)
                        .collect(Collectors.joining(" ")) + "\n");
```

```
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:

Input (stdin)

```
2
4
1 2 3 4
6
2 -1 2 3 4 -5
```

Your Output (stdout)

```
10 10
10 11
```

4.PROBLEM STATEMENT:

Given a circular integer array nums of length n, return *the maximum possible sum of a non-empty subarray of* nums.

A circular array means the end of the array connects to the beginning of the array. Formally, the next element of nums[i] is nums[(i + 1) % n] and the previous element of nums[i] is nums[(i - 1 + n) % n].

A subarray may only include each element of the fixed buffer nums at most once. Formally, for a subarray nums[i], nums[i + 1], ..., nums[j], there does not exist i <= k1, k2 <= j with k1 % n == k2 % n.

Example 1:

Input: nums = [-3,-2,-3]

Output: -2

Explanation: Subarray [-2] has maximum sum -2.

Constraints:

- n == nums.length
- $1 <= n <= 3 * 10^4$
- $-3 * 10^4 <= nums[i] <= 3 * 10^4$

SOURCE CODE:

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
```

```
        int totalSum = 0;
        int currentMax = 0;
        int maxSum = Integer.MIN_VALUE;

        int currentMin = 0;
        int minSum = Integer.MAX_VALUE;
        for (int num : nums) {
            currentMax = Math.max(num, currentMax + num);
            maxSum = Math.max(maxSum, currentMax);
            currentMin = Math.min(num, currentMin + num);
            minSum = Math.min(minSum, currentMin);
            totalSum += num;
        }
        if (maxSum < 0) {
            return maxSum;
        }

        // Circular maximum
        int circularSum = totalSum - minSum;

        return Math.max(maxSum, circularSum);
    }
}
```

OUTPUT:

Input

nums =

[1,−2,3,−2]

Output

3

Expected

3

5.PROBLEM STATEMENT:

Implement the myAtoi(string s) function, which converts a string to a 32-bit signed integer.

The algorithm for myAtoi(string s) is as follows:

1. Whitespace: Ignore any leading whitespace (" ").
2. Signedness: Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. Conversion: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. Rounding: If the integer is out of the 32-bit signed integer range [-2^{31}, $2^{31} - 1$], then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31}, and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: s = "42"

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

Step 2: "42" (no characters read because there is neither a '-' nor '+')

Step 3: "42" ("42" is read in)

Constraints:

- 0 <= s.length <= 200
- s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'.

SOURCE CODE:

```
class Solution {
    public int myAtoi(String s) {
        int i = 0;
        int sign = 1;
        int result = 0;
        while (i < s.length() && s.charAt(i) == ' ') {
            i++;
        }
        if (i < s.length() && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            if (s.charAt(i) == '-') {
                sign = -1;
            }
            i++;
        }
        while (i < s.length() && Character.isDigit(s.charAt(i))) {
            int digit = s.charAt(i) - '0';
            if (result > (Integer.MAX_VALUE - digit) / 10) {
                if (sign == 1) {
```

```
                return Integer.MAX_VALUE;
            } else {
                return Integer.MIN_VALUE;
            }
        }
        result = result * 10 + digit;
        i++;
    }
    return result * sign;
  }
}
```

OUTPUT:



6.PROBLEM STATEMENT:

You are given a string containing only the characters A and B. Your task is to transform it into a string with no matching adjacent characters by deleting zero or more characters. Return the minimum number of deletions required.

Example

- Input: $s =$ "AABAAB"
- Process: Delete A at position 0 and A at position 3 to form "ABAB".
- Output: 2 deletions.

Logic & Approach

To minimize deletions, simply iterate through the string and count every instance where a character is identical to the one immediately following it (i.e., $s[i] == s[i + 1]$).

SOURCE CODE:

```
import java.io.*;

public class Solution {

    public static int alternatingCharacters(String s) {

        int deletions = 0;

        for (int i = 0; i < s.length() - 1; i++) {

            if (s.charAt(i) == s.charAt(i + 1)) {

                deletions++;
```

```
            }
        }
        return deletions;
    }
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        int q = Integer.parseInt(bufferedReader.readLine().trim());
        for (int qItr = 0; qItr < q; qItr++) {
            String s = bufferedReader.readLine();
            int result = alternatingCharacters(s);
            System.out.println(result);
        }
        bufferedReader.close();
    }
}
```

OUTPUT:

Input (stdin)

```
5
AAAA
BBBBB
ABABABAB
BABABA
AAABBB
```

Your Output (stdout)

```
3
4
0
```

7.PROBLEM STATEMENT:

Given a string s, find the length of the longest substring without duplicate characters.

Example 1:

Input: s = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Constraints:

- 0 <= s.length <= 10^5
- s consists of English letters, digits, symbols and spaces.

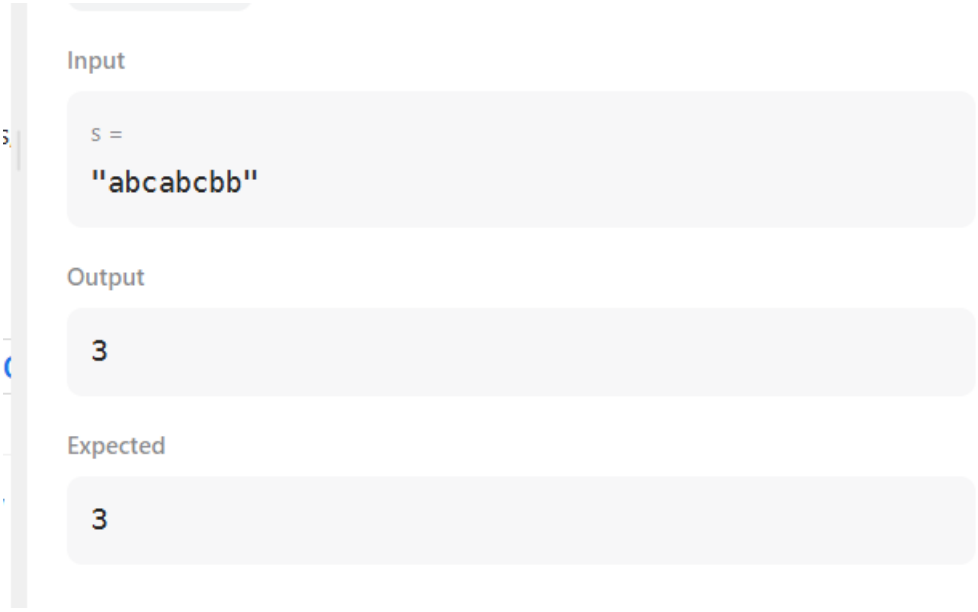
SOURCE CODE:

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int left = 0;
        int maxLength = 0;
        int[] lastIndex = new int[128];

        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            left = Math.max(left, lastIndex[ch]);
```

```
            maxLength = Math.max(maxLength, right - left + 1);
            lastIndex[ch] = right + 1;
        }
        return maxLength;
    }
}
```

OUTPUT:



8.PROBLEM STATEMENT:

Given a list of strings words and a string pattern, return *a list of* words[i] *that match* pattern. You may return the answer in any order.

A word matches the pattern if there exists a permutation of letters p so that after replacing every letter x in the pattern with p(x), we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"

Output: ["mee","aqq"]

Explanation: "mee" matches the pattern because there is a permutation {a -> m, b -> e, ...}.

"ccc" does not match the pattern because {a -> c, b -> c, ...} is not a permutation, since a and b map to the same letter.

Constraints:

- 1 <= pattern.length <= 20
- 1 <= words.length <= 50
- words[i].length == pattern.length
- pattern and words[i] are lowercase English letters.

SOURCE CODE:

```
class Solution {
```

```
    public List<String> findAndReplacePattern(String[] words, String
pattern) {
        List<String> result = new ArrayList<>();
        for (String word : words) {
            if (isMatch(word, pattern)) {
                result.add(word);
            }
        }
        return result;
    }
    public boolean isMatch(String word, String pattern) {
        int[] map1 = new int[26];
        int[] map2 = new int[26];

        for (int i = 0; i < word.length(); i++) {

            char w = word.charAt(i);
            char p = pattern.charAt(i);

            if (map1[w - 'a'] != map2[p - 'a']) {
                return false;
            }
            map1[w - 'a'] = i + 1;
            map2[p - 'a'] = i + 1;
        }
```

```
        return true;
    }
}
```

OUTPUT:

Input

words =

```
["abc","deq","mee","aqq","dkd","ccc"]
```

pattern =

```
"abb"
```

Output

```
["mee","aqq"]
```

Expected

```
["mee","aqq"]
```

9.PROBLEM STATEMENT:

Given an array of string words, return all strings in words that are a substring of another word. You can return the answer in any order.

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

Constraints:

- 1 <= words.length <= 100
- 1 <= words[i].length <= 30
- words[i] contains only lowercase English letters.
- All the strings of words are unique.

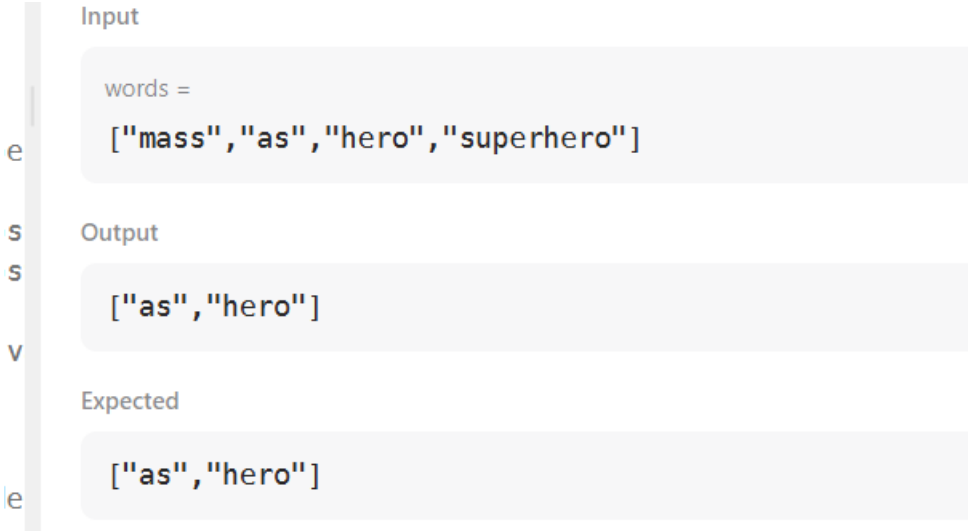
SOURCE CODE:

```
import java.util.*;

class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> result = new ArrayList<>();
        for (int i = 0; i < words.length; i++) {
            for (int j = 0; j < words.length; j++) {
```

```
            if (i != j && words[j].contains(words[i])) {
                result.add(words[i]);
                break;
            }
        }
    }
    return result;
  }
}
```

OUTPUT:



10.PROBLEM STATEMENT:

Given a text string of length n and a pattern of length m, find all starting indices where the pattern occurs in the text.
Note: You may assume that n>m.

Examples:

Input: text = "geeksforgeeks", pattern = "geeks"
Output: [0, 8]
Explanation: The string "geeks" occurs at index 0 and 8 in text.

Naive Pattern Searching Algorithm

The pattern moves over the text one position at a time and characters are compared. If all characters match, the index is stored; otherwise, the next position is checked.

SOURCE CODE:

```
import java.util.ArrayList;

public class GFG {

    static ArrayList<Integer> search(String pat, String txt)

    {

        int m = pat.length();

        int n = txt.length();

        ArrayList<Integer> ans = new ArrayList<>();

        for (int i = 0; i <= n - m; i++)

        {

            int j;
```

```
            for (j = 0; j < m; j++)
            {
                if (txt.charAt(i + j) != pat.charAt(j))
                    break;
            }
            if (j == m)
                ans.add(i);
        }
        return ans;
    }
    public static void main(String[] args)
    {
        String txt = "aabaacaadaabaaba";
        String pat = "aaba";
        ArrayList<Integer> res = search(pat, txt);
        for (int it : res)
        {
            System.out.print(it + " ");
        }
    }
}
```

OUTPUT:

Output

```
0 9 12
```

WEEK-6

1.PROBLEM STATEMENT:

Given a string S, you need to calculate the sum of similarities of S with each of its suffixes (including the string itself).

Constraints

- $1 \leq t \leq 10$ (number of test cases)
- $1 \leq |S| \leq 10^5$ (length of string S)
- S consists of lowercase English letters (a-z).

SOURCE CODE:

```
import java.io.*;
import java.util.*;


public class Solution {


   public static long stringSimilarity(String s) {
      int n = s.length();
      int[] z = new int[n];
      z[0] = n;
      long totalSimilarity = n;
      int l = 0, r = 0;
      for (int i = 1; i < n; i++) {
```

```
            if (i <= r) {
                // Use previously computed values within the current window
                z[i] = Math.min(r - i + 1, z[i - l]);
            }
            while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
                z[i]++;
            }
            if (i + z[i] - 1 > r) {
                l = i;
                r = i + z[i] - 1;
            }

            totalSimilarity += z[i];
        }

        return totalSimilarity;
    }
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        StringBuilder sb = new StringBuilder();
        while (t-- > 0) {
            String s = br.readLine().trim();
            sb.append(stringSimilarity(s)).append("\n");
```

```
        }
        System.out.print(sb);
    }
}
```

OUTPUT:

Input (stdin)

```
2
ababaa
aa
```

Your Output (stdout)

```
11
3
```

Expected Output

```
11
3
```

2.PROBLEM STATEMENT:

Given a string s, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

Input: s = "abab"

Output: true

Explanation: It is the substring "ab" twice.

Constraints:

- 1 <= s.length <= 10^4
- s consists of lowercase English letters.

SOURCE CODE:

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {
        int n = s.length();
        for (int i = 1; i <= n / 2; i++) {
            // Length must divide the string length
            if (n % i != 0) {
                continue;
            }
            String pattern = s.substring(0, i);
            String repeated = "";
            for (int j = 0; j < n / i; j++) {
```

```
            repeated = repeated + pattern;
        }
        if (repeated.equals(s)) {
            return true;
        }
    }

    return false;
  }
}
```

OUTPUT:



3.PROBLEM STATEMENT:

The Two Strings problem on HackerRank asks you to determine whether two given strings share any common substring. Since a substring can be as small as a single character, the problem simplifies to checking if the two strings share at least one common character.

Problem Summary

- Input: Two strings, s1 and s2, consisting of lowercase English letters (a-z).
- Output: Return "YES" if they share a common substring/character, or "NO" if they do not.

Examples

- Example 1:
  - s1 = "hello", s2 = "world"
  - Result: "YES" (They share common substrings like 'l', 'o').

SOURCE CODE:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static String twoStrings(String s1, String s2) {
        boolean[] visited = new boolean[26];
        // Mark characters present in s1
        for (int i = 0; i < s1.length(); i++) {
            visited[s1.charAt(i) - 'a'] = true;
        }
```

```
        // Check if any character in s2 exists in s1
        for (int i = 0; i < s2.length(); i++) {
            if (visited[s2.charAt(i) - 'a']) {
                return "YES";
            }
        }
        return "NO";
    }
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());
        for (int qItr = 0; qItr < q; qItr++) {
            String s1 = bufferedReader.readLine();
            String s2 = bufferedReader.readLine();
            String result = twoStrings(s1, s2);
            bufferedWriter.write(result);
            bufferedWriter.newLine();
        }
        bufferedReader.close();
        bufferedWriter.close();
```

```
    }
}
```

OUTPUT:

```
2  hello
3  world
4  hi
5  world
```

Your Output (stdout)

```
1  YES
2  NO
```

Expected Output

```
1  YES
2  NO
```

4.PROBLEM STATEMENT:

Given two strings s and goal, return true *if and only if* s *can become* goal *after some number of shifts on* s.

A shift on s consists of moving the leftmost character of s to the rightmost position.

- For example, if s = "abcde", then it will be "bcdea" after one shift.

Example 1:

Input: s = "abcde", goal = "cdeab"

Output: true

Constraints:

- 1 <= s.length, goal.length <= 100
- s and goal consist of lowercase English letters.

SOURCE CODE:

```
class Solution {
   public boolean rotateString(String s, String goal) {

      // Length must be same
      if (s.length() != goal.length()) {
         return false;
      }

      // Check if goal is present in s + s
```

```
        return (s + s).contains(goal);
    }
}
```

OUTPUT:

Input

s =
"m"

goal =
"f"

Output

false

Expected

false

5.PROBLEM STATEMENT:

A space explorer's ship crashed on Mars and sent a series of repetitive "SOS" messages back to Earth. Cosmic radiation altered some characters during transmission. Given the received string s, determine how many characters differ from the expected "SOS" signal sequence.

Example

- Input: s = "SOSSOT"
- Expected Signal: "SOSSOS"
- Difference: 1 character altered ('T' instead of 'S' at the end).

Function Description

Implement the marsExploration function:

- Parameter: string s — the signal received on Earth.
- Returns: int — the total count of altered letters.

SOURCE CODE:

```
import java.io.*;
class Result {
    public static String timeConversion(String s) {
        // Extract AM/PM indicator
        String period = s.substring(s.length() - 2);
        // Extract hours, minutes, and seconds
        int hours = Integer.parseInt(s.substring(0, 2));
        String restOfTime = s.substring(2, s.length() - 2);
```

```
        // Convert hours based on AM/PM logic
        if (period.equals("AM")) {
            if (hours == 12) {
                hours = 0; // 12:XX:XXAM becomes 00:XX:XX
            }
        } else { // PM
            if (hours != 12) {
                hours += 12; // 01:XX:XXPM through 11:XX:XXPM have 12 added to hours
            }
        }
        // Format hours to always have 2 digits (e.g., 0 -> "00", 7 -> "07")
        return String.format("%02d%s", hours, restOfTime);
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
        String s = bufferedReader.readLine();
        String result = Result.timeConversion(s);
        bufferedWriter.write(result);
        bufferedWriter.newLine();
        bufferedReader.close();
```

```
        bufferedWriter.close();
    }
}
```

OUTPUT:



6.PROBLEM STATEMENT:

Given two strings s and p, return an array of all the start indices of p's anagrams in s. You may return the answer in any order.

Example 1:

Input: s = "cbaebabacd", p = "abc"

Output: [0,6]

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

Constraints:

- 1 <= s.length, p.length <= $3 * 10^4$
- s and p consist of lowercase English letters.

SOURCE CODE:

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int n1 = s.length();
        int n2 = p.length();

        List<Integer> ans = new ArrayList<>();

        if (n1 < n2) return ans;
```

```
        int[] a = new int[26];
        int[] b = new int[26];

        for (int i = 0; i < n2; i++) {
            a[p.charAt(i) - 'a']++;
            b[s.charAt(i) - 'a']++;
        }

        if (Arrays.equals(a, b)) {
            ans.add(0);
        }

        for (int i = n2; i < n1; i++) {
            b[s.charAt(i) - 'a']++;
            b[s.charAt(i - n2) - 'a']--;

            if (Arrays.equals(a, b)) {
                ans.add(i - n2 + 1);
            }
        }

        return ans;
    }
}
```

OUTPUT:

Input

s =

"cbaebabacd"

p =

"abc"

Output

[0,6]

Expected

[0,6]

7.PROBLEM STATEMENT:

Given a string of lowercase letters in the range ASCII [a-z], determine the index of a character that can be removed to make the string a palindrome.

- If the string is already a palindrome or if no single character removal can make it a palindrome, return -1.
- If there are multiple valid solutions, returning any valid index is accepted.

Example

For $s =$ "bcbc":

- Removing 'b' at index 0 yields "cbc", which is a palindrome.
- Removing 'c' at index 3 yields "bcb", which is a palindrome.
- **Return:** 0 or 3.

SOURCE CODE:

```
import java.io.*;


class Result {


   public static int palindromeIndex(String s) {
      int left = 0;
      int right = s.length() - 1;


      while (left < right) {
         if (s.charAt(left) != s.charAt(right)) {
```

```
            if (isPalindrome(s, left + 1, right)) {
                return left;
            } else if (isPalindrome(s, left, right - 1)) {
                return right;
            } else {
                return -1;
            }
        }
        left++;
        right--;
    }

    return -1;
}

private static boolean isPalindrome(String s, int left, int right) {
    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}
```

```
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        for (int qItr = 0; qItr < q; qItr++) {
            String s = bufferedReader.readLine();

            int result = Result.palindromeIndex(s);

            bufferedWriter.write(String.valueOf(result));
            bufferedWriter.newLine();
        }

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:



8.PROBLEM STATEMENT:

Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

Constraints:

- 1 <= haystack.length, needle.length <= 10^4
- haystack and needle consist of only lowercase English characters.

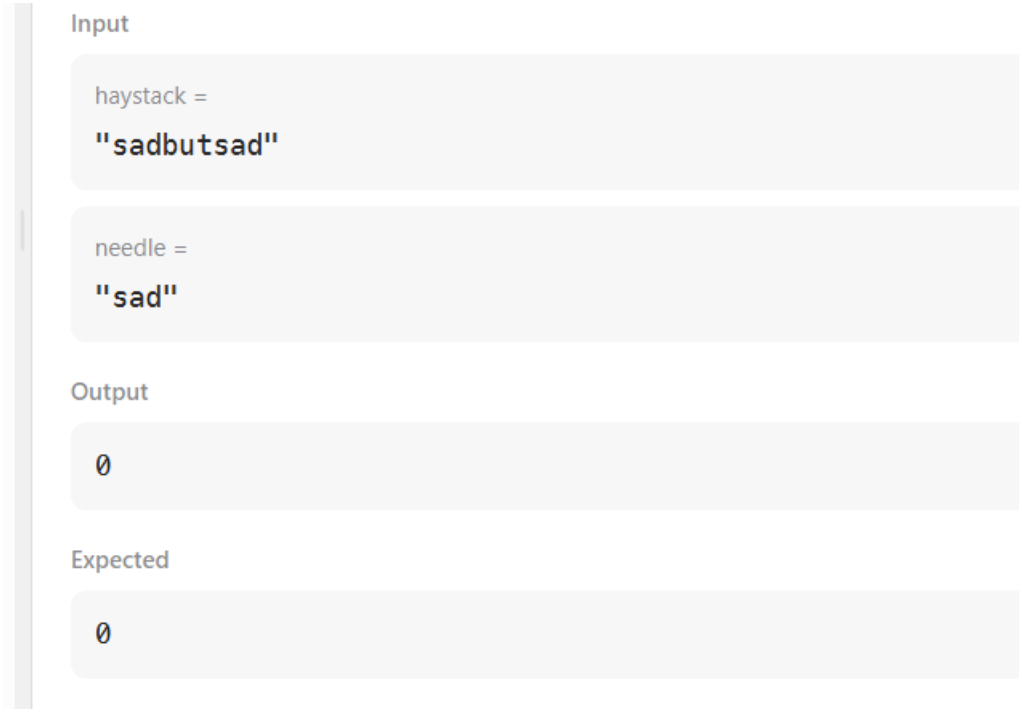
rst occurrence is at index 0, so we return 0.

SOURCE CODE:

```
class Solution {
  public int strStr(String haystack, String needle) {
 for (int i = 0; ; i++) {
   for (int j = 0; ; j++) {
     if (j == needle.length()) return i;
     if (i + j == haystack.length()) return -1;
     if (needle.charAt(j) != haystack.charAt(i + j)) break;
   }
 }
  }
```

}

OUTPUT:



9.PROBLEM STATEMENT:

Given a string s, return the longest palindromic substring in s.

A palindrome is a string that reads the same forward and backward.

Examples

Example 1:

Input: s = "babad"

Output: "bab"

"aba" is also a valid answer.

Constraints

- 1 <= s.length <= 1000
- s consists of only digits and English letters.

SOURCE CODE:

```
class Solution {
    public String longestPalindrome(String s) {
        String ans = "";

        for (int i = 0; i < s.length(); i++) {

            // Odd length palindrome
            String p1 = expand(s, i, i);

            // Even length palindrome
            String p2 = expand(s, i, i + 1);

            if (p1.length() > ans.length())
```

```
            ans = p1;

        if (p2.length() > ans.length())
            ans = p2;
    }

    return ans;
  }

  public String expand(String s, int left, int right) {

    while (left >= 0 && right < s.length()
            && s.charAt(left) == s.charAt(right)) {

        left--;
        right++;
    }

    return s.substring(left + 1, right);
  }
}
```

OUTPUT:

Input: s = "babad"

Output: "bab"

10.PROBLEM STATEMENT:

A palindrome is a string that reads the same from left to right as it does from right to left.

Given a string, S, of N lowercase English letters, we define a k-length rotation as cutting the first k characters from the beginning of S and appending them to the end of S. For each S, there are N possible k-length rotations (where $0 \leq k < N$).

Given N and S, find all N k-length rotations of S; for each rotated string, S_k, print the maximum possible length of any palindromic substring of S_k on a new line.

Input Format

- The first line contains an integer, N (the length of S).
- The second line contains a single string, S.

Constraints

- $1 \leq N \leq 5 \times 10^5$
- S consists of lowercase English letters.

SOURCE CODE:

```
import java.io.*;

import java.util.*;


public class Solution {


    public static void main(String[] args) throws IOException {
```

```
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        String nStr = br.readLine();
        if (nStr == null || nStr.trim().isEmpty()) return;

        int n = Integer.parseInt(nStr.trim());
        String s = br.readLine().trim();

        int[] ans = solve(n, s);

        StringBuilder sb = new StringBuilder();
        for (int val : ans) {
            sb.append(val).append("\n");
        }
        System.out.print(sb.toString());
    }

    private static int[] solve(int n, String s) {
        String doubleS = s + s;
        int m = doubleS.length();

        // 1. Manacher's Algorithm to find maximum palindrome radii
        StringBuilder sb = new StringBuilder("#");
        for (int i = 0; i < m; i++) {
            sb.append(doubleS.charAt(i)).append("#");
```

```
        }
        String t = sb.toString();
        int tLen = t.length();
        int[] rad = new int[tLen];

        int c = 0, r = 0;
        for (int i = 0; i < tLen; i++) {
            int mirror = 2 * c - i;
            if (i < r) {
                rad[i] = Math.min(r - i, rad[mirror]);
            }
            while (i - 1 - rad[i] >= 0 && i + 1 + rad[i] < tLen && t.charAt(i - 1
- rad[i]) == t.charAt(i + 1 + rad[i])) {
                rad[i]++;
            }
            if (i + rad[i] > r) {
                c = i;
                r = i + rad[i];
            }
        }

        // 2. Process active palindromes for each window [k, k + n - 1]
        int[] result = new int[n];

        // Priority queues to track maximum fully contained palindromes
```

```
        // pqLeft tracks palindromes bounded by the right edge of window
        // pqRight tracks palindromes bounded by the left edge of window
        PriorityQueue<int[]> pqLeft = new PriorityQueue<>((a, b) ->
Integer.compare(b[0], a[0]));
        PriorityQueue<int[]> pqRight = new PriorityQueue<>((a, b) ->
Integer.compare(b[0], a[0]));

        // Segment tree / sweep logic for fast max computation
        int[] maxPalAt = new int[m];
        for (int i = 0; i < tLen; i++) {
            int length = rad[i];
            if (length == 0) continue;

            // Map back to doubleS indices
            int center = i / 2;
            int start = (i - length) / 2;
            int end = (i + length) / 2 - 1;

            if (end - start + 1 > n) {
                // Cap palindrome length if it exceeds window size N
                int diff = (end - start + 1) - n;
                start += diff / 2;
                end -= (diff + 1) / 2;
            }
```

```
            if (start <= end) {
                maxPalAt[start] = Math.max(maxPalAt[start], end - start + 1);
            }
        }

        // Compute max palindrome length in each range [k, k + n - 1]
        for (int k = 0; k < n; k++) {
            int maxLen = 0;
            int windowEnd = k + n - 1;

            // Simple efficient scan for local max across valid indices
            for (int i = k; i <= windowEnd; i++) {
                int available = windowEnd - i + 1;
                if (maxPalAt[i] > 0) {
                    maxLen = Math.max(maxLen, Math.min(maxPalAt[i],
available));
                }
            }
            result[k] = maxLen;
        }

        return result;
    }
}
```

OUTPUT:

Input (stdin)

```
7
cacbbba
```

Your Output (stdout)

```
3
3
3
3
3
3
3
```